

Software Engineering Practical Lab Guide

Test-Driven Development with JUnit 5, Git SCM Branching & CCB Workflows

Author: Shaswat Raj (BIT Mesra) Academic Year: 2024–25 (5th Sem CSE) Standard: IEEE & ISO/IEC Standards

Lab Experiment 1: Automated Unit Testing with JUnit 5 & Mockito

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

public class BankAccountTest {
    private BankAccount account;

    @BeforeEach
    void setUp() {
        account = new BankAccount("ACC-1001", 1000.0);
    }

    @Test
    @DisplayName("Deposit positive amount increases balance")
    void testValidDeposit() {
        account.deposit(500.0);
        assertEquals(1500.0, account.getBalance(), 0.001);
    }

    @Test
    @DisplayName("Withdrawal exceeding balance throws InsufficientFundsException")
    void testOverdrawThrowsException() {
        assertThrows(InsufficientFundsException.class, () -> {
            account.withdraw(2000.0);
        });
    }
}
```

Lab Experiment 2: Git Software Configuration Management (SCM) & Branching

```
# 1. Create and switch to feature branch
git checkout -b feature/user-auth-jwt

# 2. Stage changes and create verified commit linked to Requirement ID
git add src/main/java/com/auth/JwtService.java
git commit -m "feat(auth): implement JWT token generation [REQ-AUTH-101]"

# 3. Rebase on main and push to remote
git checkout main && git pull origin main
git checkout feature/user-auth-jwt
git rebase main
git push origin feature/user-auth-jwt
```

Lab Experiment 3: StarUML Object-Oriented Modeling & Reverse Engineering

1. Launch StarUML → Select Model → Add Diagram → Class Diagram.
2. Create Classes: User, Account, Transaction with attributes and visibility annotations (+, -, #).
3. Draw Composite and Aggregation relationships with multiplicity bounds (1..*, 0..1).
4. Export Java code stubs automatically using the StarUML Java Extension generator!

Lab Experiment 4: Mockito Mocking in Spring Boot Service Testing

```
@ExtendWith(MockitoExtension.class)
public class OrderServiceTest {
    @Mock private PaymentGateway paymentGateway;
```

```

@Mock private InventoryRepository inventoryRepo;
@InjectMocks private OrderService orderService;

@Test
void testProcessOrderSuccess() {
    when(inventoryRepo.isInStock("SKU-100", 2)).thenReturn(true);
    when(paymentGateway.charge(100.0)).thenReturn(new PaymentResult(true, "TXN-999"));

    Order order = new Order("SKU-100", 2, 100.0);
    OrderResult result = orderService.processOrder(order);

    assertTrue(result.isSuccess());
    verify(paymentGateway, times(1)).charge(100.0);
}
}

```

Lab Experiment 5: Continuous Integration (CI) Pipeline with GitHub Actions

```

name: Java CI with Maven & JUnit 5
on: [push, pull_request]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          java-version: '17'
          distribution: 'temurin'
      - name: Build with Maven
        run: mvn clean test --batch-mode
      - name: Publish Test Results
        uses: EnricoMi/publish-unit-test-result-action@v2
        if: always()
        with:
          files: target/surefire-reports/*.xml

```

Lab Experiment 6: Static Code Quality Analysis with SonarQube & SpotBugs

```

# Run static analysis and generate technical debt reports
mvn clean compile spotbugs:spotbugs
mvn sonar:sonar -Dsonar.projectKey=banking-portal -Dsonar.host.url=http://localhost:9000

```

Lab Experiment 7: Continuous Deployment (CD) with Docker & Kubernetes

```

FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/banking-service-1.0.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]

```

Lab Experiment 8: Automated Security Scanning with OWASP ZAP & Dependency-Check

```

# Scan Java dependencies for known CVE vulnerabilities
mvn org.owasp:dependency-check-maven:check

```

Lab Experiment 9: Automated Load Testing with Apache JMeter

```

# Run headless performance benchmark with 1000 concurrent threads
jmeter -n -t test_plan.jmx -l results.jtl -e -o ./report_dashboard

```

Lab Experiment 10: Automated Containerized Microservices Testing with Testcontainers

```
@Testcontainers
class UserRepositoryTest {
    @Container
    static PostgreSQLContainer postgres = new PostgreSQLContainer<>("postgres:15-alpine");

    @Test
    void testSaveAndFindUser() {
        // Runs real isolated database container automatically in CI!
    }
}
```

Lab Experiment 11: End-to-End API Testing with Postman & Newman CI

```
# Run automated API regression test suite in headless continuous integration
newman run postman_collection.json -e staging_env.json --reporters cli,htmlextra
```

Lab Experiment 12: Performance & Load Benchmark Analysis

```
# Run Artillery HTTP load test simulating 5000 virtual users
artillery run load_test_scenario.yml --output load_report.json
artillery report load_report.json
```